

Frontend Engineer Technical Assessment

Overview

Build a polished recruitment web experience using **Vite + React**. The goal is to simulate a real product surface: a **main recruitment page** (discover + browse candidates/roles) and a **candidate profile** page.

Timebox: 4–6 hours

Max routes/pages: 2

Stack: Vite + React (TypeScript preferred)

1) Requirements

1.1 Project Setup (Required)

- Create the project using **Vite**.
- Use **React** (TypeScript preferred; JS acceptable).
- Use a router (React Router or similar).
- Must run with:
 - `npm install`
 - `npm run dev`
- Provide a clean repo structure (components, pages, hooks/services, styles).

1.2 Pages (Max 2 routes)

You must implement **exactly these two routes**:

1. **Main Recruitment Page:** `/`
2. **Candidate Profile Page:** `/candidate/:id`

A profile modal is allowed, but only if it still preserves the 2-route maximum.

1.3 Data (Required)

Use either:

- **Mock data (recommended)** via `/src/data/candidates.json` (20–40 candidates) with simulated API latency, OR
- A public API that you transform into candidates.

Minimum candidate fields

Your candidate entity must support:

- `id`
- `fullName`
- `headline` (e.g., “Frontend Engineer | React | UI Systems”)
- `location`
- `yearsOfExperience` (or `seniority`)
- `skills` (array)

- **availability** (e.g., “Immediate”, “2 weeks”, “1 month”)
- **updatedAt**
- **status** (e.g., “Open to work”, “Interviewing”, “Hired”, etc.)
- **score** (0–100 or 1–5)

You may add: portfolio links, education, languages, salary expectations, notes, etc.

1.4 Main Recruitment Page Requirements (✓)

This page should feel like a real “recruitment landing + directory” page.

A) Header / Hero (Required)

Include a top section that communicates:

- Product/company name (your choice)
- A short tagline
- A primary call-to-action (e.g., “Browse Candidates”, “Explore Talent”)
- Optional: a small “stats” strip (e.g., # candidates, top skills, locations)

B) Candidate Directory (Required)

Display candidates in a designed layout (grid cards or list—must be visually intentional).

Each candidate card must show:

- Name
- Headline
- Location
- Experience or seniority
- 3–6 skill chips
- A visible “status” badge
- A clear action: “View Profile”

C) Search / Filter / Sort (Required)

Provide controls that allow meaningful exploration:

Search

- Search by name OR skills OR headline

Filter (at least 2)

- Location
- Seniority / years of experience
- Skill tag
- Availability
- Status

Sort (at least 1)

- Recently updated
- Highest score
- Most experience

Also include:

- Results count (“24 candidates”)
- Clear “active filters” UI (chips/tags)
- One-click “Reset all”

D) States (Required)

Design and implement:

- **Loading state** (skeletons recommended)
- **Error state** (with retry)
- **Empty state** (no matches)

E) Responsiveness & Polish (Required)

- Mobile + desktop layouts must work well.
- Accessible basics: labeled inputs, focus states, keyboard-friendly controls.
- Visual consistency: spacing, typography, colors, reusable components.

1.5 Candidate Profile Page Requirements (/candidate/:id)

The profile must feel like a real candidate page.

A) Profile Header (Required)

- Candidate name + headline
- Status badge

- Location, experience (quick summary row)
- Primary action(s) (see below)

B) Profile Sections (Required)

Include at least these sections:

1. **About / Summary** (short paragraph)
2. **Skills** (chips + optionally proficiency)
3. **Experience / Projects** (timeline or cards—your choice)
4. **Metadata panel** (structured key/value: availability, updatedAt, score, etc.)

C) Actions (Required)

Implement at least **two** actions that update UI state:

- “Shortlist” / “Save”
- “Reject”
- Optional: “Move Status” dropdown

Actions must:

- Immediately reflect on the profile UI
- Be reflected back on the main page when you navigate back (state persistence)

D) Navigation (Required)

- Clear way back to / (Back button or breadcrumb)
- Preserve browsing context if possible (filters/search retained)

2) Preferences (Not required, but strongly valued)

UI/UX Preferences

- A cohesive theme and consistent typography scale
- Excellent spacing and alignment
- Thoughtful micro-interactions (hover/pressed/focus, subtle transitions)
- Great empty states (friendly + helpful)
- Responsive behavior that feels designed (not “shrunk desktop”)

Engineering Preferences

- TypeScript with clean, meaningful types
- URL-driven state for search/filter/sort (query params)
- A small data layer ([services/](#) or [api/](#)) with clear separation
- Good component reuse (Button, Badge, Card, Tag, Input, etc.)
- Accessibility:
 - keyboard navigation for key actions
 - focus management (especially if modal)
 - semantic HTML
- Performance considerations (memoization where it matters)

Quality Preferences

- Clean README with decisions + tradeoffs
- At least 3 tests (if you choose testing as a focus)
- Error boundary or graceful error handling
- Reasonable lint/format setup (optional)

3) Technical Focus (Choose at least 3)

Pick at least **3** from below and implement well (call out in README):

1. URL-driven state (query params)
2. State management approach (Context/Zustand/Redux/local)
3. Data caching (React Query/SWR/simple cache)
4. Accessibility enhancements (ARIA, focus, keyboard)
5. Testing (Vitest + Testing Library)
6. Reusable component system + design tokens
7. Error boundary + fallback UI
8. Strong type safety (+ optional runtime validation)

4) Deliverables

Submit:

- Repo link (preferred) or zip

Include a **README** (You can use AI for the readme file) with:

- Setup instructions
- Feature checklist
- Which technical focuses you implemented
- Data approach (mock/API)
- Tradeoffs and next improvements